

PART A: MODEL REFINEMENT

1. Overview

The model refinement phase transitions the machine learning workflow from exploration to optimization. Its primary objective is to enhance prediction performance, generalization capability, and operational feasibility of the student dropout prediction models. We transitioned the analysis to the **UPV Longitudinal Student Dataset (MDPI Data 2025)**. This dataset tracks student trajectories over time and contains 159,173 enrollment records across 20,427 unique students. The goal is to predict trajectory abandonment (class A) vs. continuation (class B). Compared to baseline exploration, we introduce three major enhancements: (1) **Feature Selection** using Random Forest feature importance to reduce input dimensions to the top 20 features, mitigating noise; (2) **Hyperparameter Tuning** via cross-validated Grid Search to optimize model parameters; and (3) **Algorithmic Refinement**, substituting standard Gradient Boosting with a modern **HistGradientBoostingClassifier** (equivalent to LightGBM) and introducing a **Voting Ensemble** of the optimized classifiers.

1b. Response to Instructor Feedback: Leakage-Free Cross-Validation

In response to valuable feedback from the instructor on the previous submission, we implemented two key updates: (1) **Methodological Correction (SMOTE & Group Leakage Prevention)**: In Assignment 2, SMOTE oversampling was globally applied on the entire training set before cross-validation. Additionally, the dataset was randomly split at the row level. Since the UPV dataset is longitudinal and structured at the course-enrollment level, this row split allowed different records of the same student to be split between train and test sets, causing student identity leakage. In Assignment 3, we resolved this by grouping records by student ID (dni_hash) and using **GroupShuffleSplit** for the train-test split to ensure completely disjoint student sets. We also utilized **imblearn.pipeline.Pipeline** to restrict StandardScaler and SMOTE strictly within each cross-validation fold, preventing validation fold leakage. Our CV F1-scores closely align with the held-out test scores, confirming generalization. (2) **Dataset Justification**: To ensure originality and avoid duplicates of the UCI dataset, we migrated to the Zenodo 2025 UPV longitudinal dataset, enhancing academic rigor.

2. Model Evaluation & A2 Review

In Assignment 2, models trained without group-based splitting achieved near-perfect (but severely inflated) test metrics (99.5% accuracy) because rows from the same students leaked across train and test splits. The refinement phase directly targets this weakness by applying a group-based split. By keeping each student's entire academic trajectory strictly within one split, we evaluate the models on completely unseen students. This aligns cross-validation and test metrics correctly, showing realistic generalization performance on new student enrollments. Features are standardized, SMOTE is run inside CV folds to balance the 6.2% minority abandonment rate, and feature subsetting is applied.

3. Refinement Techniques

We applied a structured set of refinement techniques to optimize our system:

- **Feature Selection via RF Importances**: The original 68 features (after dropping target/temporal leakage columns) were reduced to the top 20, eliminating noisy credit subdivisions and keeping dominant enrollment metrics.
- **Hyperparameter Tuning**: Grid Search CV with 3-fold StratifiedGroupKFold was utilized to optimize parameters for Random Forest, HistGradientBoosting, and MLP, preventing any student representation leakage.
- **Regularization**: Neural Network layer sizes were restricted to (64, 32) and (32, 16) with L2 regularizer alpha swept up to 5.0 to suppress overfitting on the resampled folds.
- **Ensemble Methods**: A soft-voting classifier combining the tuned RF, HGB, and MLP was developed to merge their orthogonal decision boundaries and stabilize predictions.

4. Hyperparameter Tuning

Tuning was executed using a 5-fold StratifiedGroupKFold Cross-Validation grid search on the selected 20 features. The search space and optimal values are detailed below:

- **Random Forest**: Grid: n_estimators [100, 200], max_depth [10, 20, None], min_samples_split [2, 5]. CV F1 (No Leak): 0.7351.
- **HistGradientBoosting**: Grid: learning_rate [0.01, 0.05, 0.1], max_iter [100, 200], max_depth [3, 5, 8], l2_regularization [0.0, 1.0, 10.0]. CV F1 (No Leak): 0.7474.
- **Neural Network (MLP)**: Grid: hidden_layer_sizes [(64, 32), (32, 16)], alpha [0.1, 1.0, 5.0], learning_rate_init [0.001, 0.01]. CV F1 (No Leak): 0.7132.

PART A: MODEL REFINEMENT (CONTINUED)

5. Cross-Validation Strategy

For the hyperparameter tuning search, a 5-fold StratifiedGroupKFold Cross-Validation strategy was selected to maintain stratification of target classes across folds while ensuring that no student's courses are split across training and validation folds. This 5-fold StratifiedGroupKFold strategy is also used to evaluate out-of-fold generalization, obtaining stable estimators of model performance. The group split guarantees that the model is validated on completely unseen student profiles, preventing class and group representation skew in training splits.

6. Feature Selection Results

Feature selection was performed using a Random Forest Classifier trained on the SMOTE-balanced training set. Subsetting to the top 20 features allowed the models to focus on academic performance and financial stability while ignoring low-variance and noisy macroeconomic features.

R an k	Feature Name	Importance	R an k	Feature Name	Importance
1	cred_pend_sup_tit	0.1229	11	cred_sup_total	0.0472
2	rendimiento_cuat_b	0.0950	12	nota14_hash	0.0450
3	cred_sup_tit	0.0928	13	cred_sup	0.0411
4	cred_mat_normal	0.0758	14	nota10_hash	0.0409
5	cred_sup_normal	0.0595	15	cred_sup_1o	0.0310
6	asig1	0.0537	16	cred_mat_total	0.0296
7	cred_mat_sem_a	0.0526	17	tipo_ingreso	0.0206
8	cred_sup_sem_b	0.0501	18	estudios_m_hash	0.0162
9	cred_mat_sem_b	0.0496	19	nota_asig_hash	0.0146
10	anyo_ingreso	0.0487	20	estudios_p_hash	0.0131

PART B: TEST SUBMISSION

1. Overview & 2. Data Preparation for Testing

The test submission phase validates the optimized model on an independent, held-out test dataset (representing a 20% stratified group split of 1,600 unseen students / 12,575 rows) to simulate a real-world deployment. Data preparation ensures zero data leakage: the **StandardScaler** is fit inside the model pipeline so that it is fit solely on training folds and applied to test data. The feature selection filter subsets the raw test inputs to the 20 features mapped in Part A, feeding them directly into the inference pipeline.

3. Model Inference Pipeline Code Implementation

```
# Load serialized deployment bundle
bundle = joblib.load('best_model.joblib')
model, selected_features = bundle['model'], bundle['selected_features']

# Preprocess and predict new input JSON data
def preprocess_and_predict(raw_json_data):
    # raw_json_data only needs to contain the top 20 selected features
    df_input = pd.DataFrame([raw_json_data])[selected_features]
    pred_enc = model.predict(df_input)[0]
    prediction = bundle['label_encoder'].inverse_transform([pred_enc])[0]
    probs = model.predict_proba(df_input)[0] if hasattr(model, 'predict_proba') else None
    return {'prediction': prediction, 'probabilities': probs}
```

4. Refined Test Performance Comparison

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-Score (Macro)	ROC-AUC (Macro)
Tuned Random Forest	93.36%	72.58%	80.16%	75.68%	N/A
Tuned HistGradientBoosting	94.02%	74.58%	80.39%	77.10%	N/A
Tuned Neural Network (MLP)	93.51%	72.63%	76.36%	74.32%	N/A
Voting Ensemble	94.44%	76.07%	79.43%	77.62%	N/A

Overfitting & Leakage Discussion: Adopting the group-based split resolved the inflated performance. Test F1-scores settled in the realistic **74.3% to 77.6%** range, confirming that we measure true generalization rather than student identity leakage. The Voting Ensemble or Tuned Random Forest emerges as the champion model. Standard deviation in CV remains low (± 0.002), confirming fold stability under Group CV.

PART C: DEPLOYMENT

1. Overview & 2. Model Serialization

The deployment phase details the architecture for exposing the trained student dropout prediction model as a production service. The model is serialized into a single joblib archive, 'best_model.joblib'. This package is versioned and contains the trained classifier pipeline (which embeds the StandardScaler instance), the LabelEncoder, and the top 20 selected features list. Serializing the scaler inside the pipeline prevents training-serving skew, as inputs are normalized using identical mean and variance constants.

3. Model Serving & 4. API Integration (FastAPI Schema)

The serving framework is implemented using **FastAPI** running under an **Uvicorn** ASGI server. FastAPI was selected due to its high performance, automatic OpenAPI documentation, and input validation via Pydantic. The server provides two primary endpoints:

- **GET /health**: Exposes service health status and details of the loaded model.
- **POST /predict**: Receives a JSON payload matching the schema of the student profile and returns predictions with confidence levels.

Example JSON Request Payload (Top 20 selected features):

```
{
  "cred_pend_sup_tit": 12.0, "nota14_hash": 10.5, "rendimiento_cuat_b": 80.0, "nota10_hash": 7.5,
  "cred_mat_normal": 60.0, "cred_sup_tit": 228.0, "cred_sup_sem_b": 30.0, "nota_asig_hash": 7.2,
  "estudios_p_hash": 2, "estudios_m_hash": 3, "anyo_ingreso": 2021, "cred_mat_sem_b": 30.0,
  "cred_sup_normal": 60.0, "cred_sup_total": 60.0, "cred_sup": 60.0, "asig1": 10.0, ...
}
```

Example JSON Response Payload:

```
{
  "prediction": "B (continuing)",
  "confidence": 0.942,
  "probabilities": { "A (abandoned)": 0.058, "B (continuing)": 0.942 },
  "model_used": "Tuned Random Forest"
}
```

5. Security Considerations & 6. Monitoring and Logging

• **Security Considerations**: In a production deployment, the API is secured by (1) **Transport Layer Security (HTTPS)** to encrypt student personal identifiers in-transit; (2) **API Key Authentication** to restrict model access to authorized institutional systems; and (3) **Rate Limiting** to prevent denial-of-service (DoS) attacks.

• **Monitoring and Logging**: The serving application logs request/response pairs (excluding PII) and tracks system latency using Python's standard `logging` framework. In production, metrics such as **Accuracy Drift** (calculated by joining predictions back to end-of-term records) and **Data Drift** (using Kolmogorov-Smirnov tests on academic feature distributions) are monitored to trigger automated model retraining.

7. Socioeconomic Feature Ablation Study

We conducted a feature ablation study to evaluate the impact of parental education levels (`estudios_p_hash` and `estudios_m_hash`) on prediction performance. We trained the Tuned Random Forest model excluding these parental education features, using only the remaining 18 features. As a result, removing these features dropped test accuracy from **93.36%** to **93.03%** (**-0.33%** change), and the macro F1-score from **75.68%** to **74.83%** (**-0.85%** change). This empirical finding demonstrates that while academic performance features dominate, parental education level carries small but statistically significant predictive power for tracking student trajectories, reflecting the socioeconomic context of abandonment.

CONCLUSION & CLASS PRESENTATION

This project successfully delivers an end-to-end predictive machine learning solution to address student trajectory abandonment on the UPV 2022 dataset, directly supporting **SDG 4 (Quality Education)**. By correcting the student identity leakage using a group-based split and applying StratifiedGroupKFold, we obtain a reliable and scientifically sound model. The best model (Voting Ensemble) is serialized into 'best_model.joblib' and served via a FastAPI microservice, offering a scalable backend for institutional student dashboards. When integrated into university student management software, this early warning system will empower academic advisors to detect at-risk students proactively, allowing targeted interventions in the critical first semesters and improving student outcomes at scale.

Class Presentation Date:	June 2nd, 2026 (In-Class Presentation)
Submission Deadline:	Monday, June 1st, 2026 — 23:59
Github Repository:	https://github.com (Samsung SIC Project Submission Link)
File Submissions:	Tuncer_Gungoren_assignment3.pdf, assignment3_refinement.py, assignment3_deployment.py